

SMART USB

Activity Detection and Access Control System Project Planning & Implementation Report

1. Project Overview

Smart USB is a planned desktop-based security application designed to monitor USB device connections and provide an access-control workflow before allowing file access. The project will detect a connected USB device, collect available device information, record activity, and provide an owner-controlled approval mechanism. The current implementation is being developed as a Python project in Visual Studio Code.

Important: This document describes the features and implementation that we have planned so far. It is a project-planning report, not a claim that every feature has already been implemented or tested.

2. Main Objectives

- Detect USB storage devices when they are connected to the computer.
- Identify the USB device using information available from Windows, such as device name, manufacturer and serial number where available.
- Place the device into a restricted/pending state until the access decision is made.
- Provide an owner approval workflow with direct approval or password-protected approval.
- Record connection, access and file activity in a local database.
- Provide a dashboard/interface for viewing USB status and activity.
- Generate reports from stored activity data.
- Add owner notification by email as part of the planned access-request workflow.

3. Planned User Workflow

Step	Planned action
1	USB device is connected to the computer.
2	The application detects the USB device.
3	Device information and connection time are collected.
4	The device is placed in a restricted/pending access state.
5	An access request is presented to the owner; email notification is planned.
6	Owner chooses either Direct Access Approval or Password Protected Access.
7	If approved, the application records the decision and changes the device status accordingly.
8	If password protection is selected, the user must enter the security password; failed attempts are logged.
9	USB/file activity is recorded for monitoring and reporting.
10	Owner can review logs and generate reports from the dashboard.

4. Access-Control Options

A. Direct Access Approval

The owner can approve the detected USB device directly. The planned result is that the device changes from restricted/pending status to an approved/unlocked state and normal file access is permitted.

B. Password Protected Access

The owner can require a security password before USB access is granted. The system is planned to track failed attempts. A future/extended part of the workflow includes a forgot-password option using OTP sent to the registered email or phone number.

5. Owner Notification Plan

When a new USB access request is detected, the planned system will notify the owner by email. The owner email may be configured by the user rather than hard-coded. The project will provide an option to connect or configure the owner's email account. The exact Gmail authentication method will be selected during implementation so that account credentials are not stored insecurely.

6. Planned System Architecture

Component	Purpose
USB Detection	Detect USB connection/removal and collect device information.
Authentication	Manage security password, access decisions and failed attempts.
Monitoring	Monitor relevant USB/file activity and create log records.
Database	Store USB devices, access logs and file activity.
Backend	Coordinate detection, authentication, database and application logic.
Frontend / Dashboard	Show USB status, requests, logs and controls to the owner.
Notification	Send planned access-request notifications to the owner.
Reports	Produce readable activity reports from stored data.

7. Planned Technology Stack

Technology / Tool	Planned use
Python 3.x	Main programming language.
Visual Studio Code	Development environment.
Windows / WMI + pywin32	Planned approach for Windows USB/device detection.
SQLite	Local database for project records.
Flask	Planned lightweight backend/web API layer.
HTML/CSS/JavaScript	Planned frontend/dashboard.
watchdog	Planned file-system monitoring component where appropriate.
bcrypt / hashing	Planned secure password handling.
ReportLab	Planned PDF report generation.

8. Planned Project Folder Structure

SmartUSB/

backend/

app.py — planned backend entry point

usb_detection/ — USB detection logic

authentication/ — password and access-control logic

monitoring/ — activity/file monitoring logic

database/

database.py — SQLite database functions

frontend/ — planned web/dashboard interface

desktop/ — planned desktop-side integration

docs/ — project documentation

logs/ — generated/application logs

reports/ — generated reports

9. Planned Database Design

Table	Planned information
usb_devices	USB/device identity, manufacturer/serial information where available, first/last connection and status.
access_logs	Access requests, decisions, timestamps, password-attempt information and relevant user/action details.
file_logs	Relevant file activity associated with the monitored USB device, subject to the monitoring scope implemented.

10. Planned Dashboard Pages

- Dashboard — overall USB status, recent events and security state.
- USB Devices — connected/known devices and device details.
- Access Requests — pending requests and owner approval controls.
- Authentication — password configuration and failed-attempt information.
- Activity Logs — USB/file activity history with timestamps.

- Reports — generate and view activity/security reports.
- Settings — owner email and other application configuration.

11. Implementation Roadmap

1. Environment setup — verify Python interpreter in VS Code and install required packages.
2. Backend foundation — create the Flask application and project configuration.
3. Database — create SQLite database and required tables.
4. USB detection — implement Windows USB detection and device-information collection.
5. Access control — implement pending/approved/restricted states and password verification.
6. Monitoring — add the planned activity/file monitoring and logging.
7. Notification — implement configurable owner email notification.
8. Frontend — build the dashboard and access-request screens.
9. Reports — generate PDF activity reports.
10. Integration testing — connect all components and test USB insertion/removal, approval, denial, password attempts and logging.
11. Final testing — verify reliability, security and usability before demonstration.

12. Security Considerations

- Passwords should not be stored as plain text; use a secure password-hashing method.
- Email credentials should not be hard-coded into source files.
- Owner configuration should be protected from unauthorized changes.
- Access decisions and failed authentication attempts should be logged.
- The system should clearly distinguish detected, pending, approved and restricted states.
- USB/file monitoring should be limited to the project's intended scope and tested carefully to avoid unintended system changes.
- The final implementation should be tested on the target Windows environment because USB/device APIs can vary by system.

13. Expected Final Demonstration

For the project demonstration, the intended sequence is: start the Smart USB application → connect a USB storage device → show device detection → show restricted/pending status → show owner access request → demonstrate direct approval or password protected access → show the resulting status → display the activity log → generate a report. The demonstration should use a test USB device and should not rely on unimplemented functionality.

14. Current Development Status

Area	Status
Python installation	Installed and verified in the development environment.
VS Code project	SmartUSB project folder opened.

Area	Status
Workspace trust	Trusted folder has been enabled.
Project folders	Initial backend/database/desktop/docs/frontend/logs/reports structure created.
Required packages	Initial Python package installation has been started; further dependencies will be installed as implemented
Application implementation	Next stage: build and connect the backend components step by step.

Prepared as a planning document for the Smart USB project. Features may be refined during implementation after testing the Windows environment and selected libraries.